

14

Causal Discovery and Machine Learning – Advanced Deep Learning and Beyond

Welcome to *Chapter 14*!

We're inevitably moving towards the end of our book, but we still have something to learn!

In the previous chapter, we introduced four families of causal discovery models: constraint-based, score-based, functional, and gradient-based. Each of the families and methods that we discussed came with unique strengths and unique limitations.

In this chapter, we'll introduce methods and ideas that aim to solve some of these limitations. We'll discuss an advanced deep learning causal discovery framework, **Deep End-to-end Causal Inference (DECI)**, and implement it using the Microsoft open source library Causica and PyTorch.

We'll see how to approach data with hidden confounding using the **fast causal inference (FCI)** algorithm and introduce other algorithms that can be used in similar scenarios.

After that, we'll introduce two frameworks that allow us to combine observational and interventional data: ENCO and ABCI.

We'll close this chapter with a discussion of the challenges and open areas for improvement in causal discovery.

By the end of this chapter, you'll understand the basic theory behind DECI and will be able to apply it to your own problems. You will understand when the FCI algorithm might be useful and will be able to use it, including adding expert knowledge.

Finally, you will have a good understanding of how to extend observational causal discovery to interventional data and what challenges we face when applying causal discovery methods to real-world problems.

In this chapter, we cover the following topics:

- Deep End-to-end Causal Inference (DECI)
- Causal discovery under hidden confounding
- The FCI algorithm
- Causal discovery with interventions
- The real-world applicability of causal discovery and open challenges

Advanced causal discovery with deep learning

Xun Zheng and colleagues' *DAGs with NO TEARS* paper (Zheng et al., 2018), which we introduced in the previous chapter, ignited excitement in the causal discovery community and inspired a whole new line of research on gradient-based methods.

The fact that graph search could be carried out using continuous optimization opened up a path for integrating causal discovery with techniques coming from other deep learning areas.

One example of a framework integrating such techniques into the realm of causal discovery is **DECI** – a **deep learning end-to-end causal discovery and inference** framework (Geffner et al., 2022).

DECI is a flexible model that builds on top of the core ideas of the *NO TEARS* paper. It works for non-linear data with additive noise under minimality and no hidden confounding assumptions.

In this section, we'll discuss its architecture and major components and apply it to a synthetic dataset, helping the model to converge by injecting expert knowledge into the graph.

This section will have a slightly more technical character than some of the previous chapters. We'll focus on gaining a deeper understanding of the framework and code.

This focus will help us get a grasp of how advanced methods in causal discovery might be designed and how to implement them using lower-level components.

This is also an opportunity to get a taste of how causal algorithms can be implemented using deep learning frameworks such as PyTorch.

From generative models to causality

In recent years, generative models have made their way to the headlines. Models such as ChatGPT, DALL-E 2, and Midjourney have astonished the general public with their capabilities, sparked broad interest, and inspired a number of debates on the future of AI and even humanity itself; but generative models are far from being new.

The ideas behind generative models can be traced back to the 19th century and the early works of Adolphe Quetelet and – a bit later – Karl Pearson (McLachlan et al., 2019).

Contemporary causal discovery has multiple connections to generative modeling. For instance, DECI uses **causal autoregressive flow** (Khemakhem et al., 2021), while the **Structural Agnostic Model (SAM)** (Kalainathan et al., 2022) leverages **generative adversarial networks (GANs)** (Goodfellow et al., 2020).

Let's briefly review the connection between autoregressive flows and causal models.

Looking back to learn who you are

Autoregressive flows (Kingma et al., 2016) are a subset of the **normalizing flows** framework – a set of techniques traditionally used in variational inference in order to efficiently learn complex distributions by transforming simpler ones.

In particular, autoregressive flows estimate the distribution of variable X_k as a function of the variables *preceding* it.

If we want to express one variable as a function of the variables preceding it, we first need to *order* these variables somehow.

A setting where variables are ordered and their value only depends on the variables that are *preceding* them resembles a **structural causal model (SCM)**, where nodes' values only depend on the values of their parents.

This insight has been leveraged by Ilyes Khemakhem and colleagues (Khemakhem et al., 2021), who proposed an autoregressive flow framework with causal ordering of variables (**CAREFL**).

DECI further builds on this idea, by modeling the likelihood of data given the graph in an autoregressive fashion.

Moreover, the model learns the posterior distribution over graphs rather than just a single graph. In this sense, DECI is Bayesian. This architectural choice allows for a very natural incorporation of expert knowledge into the graph. We will see this in action later in this section.

DECI's internal building blocks

At the heart of DECI lies the same core idea that enabled NO TEARS' continuous optimization: the DAG-ness score. Although we introduced this in the previous chapter, let's refresh the definition here:

$$\mathcal{R}(A) = \text{tr}(e^{A \odot A}) - d$$

In the preceding formula, note the following:

- A is the adjacency matrix of a graph, G
- e is a matrix exponential
- $\text{tr}(\cdot)$ is a trace of a matrix
- d is the number of nodes in a graph, represented by A
- $\odot$ is the Hadamard (element-wise) product

The role of $\mathcal{R}(A)$ is to ensure that we focus our search on **directed acyclic graphs (DAGs)**. $\mathcal{R}(A)$ is equal to 0 *if and only if* a graph represented by the matrix A is a DAG.

In other words, minimizing this component during training helps us make sure that the recovered model will not contain cycles or bi-directional edges.

The DAG-ness score is one of the components that constitute DECI's loss function; but it's not the only one. A second important component is the **sparsity score**.

The sparsity score is defined using the squared **Frobenius norm** of the adjacency matrix. Formally, this is represented as follows:

$$\mathcal{S}(A) = \|A\|_F^2$$

The role of the sparsity score is to (heuristically) promote the minimality of the solution. In other words, $\mathcal{S}(A)$ pushes the graph to follow the **minimality assumption** (for a refresher on minimality, refer to *Chapter 5* and/or *Chapter 13*).

Frobenius norm

The Frobenius norm is a matrix norm (<https://bit.ly/MatrixNorm>) defined as the square root of the **sum** of squares of absolute values of matrix entries:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

When A is an unweighted adjacency matrix, the Frobenius norm measures the sparsity of the matrix. The sparser the matrix, the more zero entries it has.

Note that if the matrix A is real-valued, the absolute value function $| \cdot |$ is redundant (squaring will make the result non-negative anyway). This only matters for complex-valued matrices (which we're not using here, but we include the absolute value operator for completeness of the definition).

The DAG-ness score and sparsity score are used to define three terms used in the model's loss function:

- $\alpha \mathcal{R}(A)$
- $\rho \mathcal{R}(A)^2$
- $\lambda \mathcal{S}(A)$

Each term is weighted by a dedicated coefficient: α , ρ , and λ , respectively.

While the latter coefficient is kept constant over the training, the first two are gradually increased.

The idea here is not to overly constrain the search space in the beginning (even if early graphs are not DAGs) in order to allow the algorithm to explore different trajectories. With time, we increase the values of α and ρ , effectively limiting the solutions to DAGs only.

The updates of these parameters are carried out by the scheduler of the **augmented Lagrangian optimizer** (<https://bit.ly/AugmentedLagrangian>).

The initial values of α , ρ , and λ are set by the user (they play the role of hyperparameters) and can influence the trajectory of the algorithm. In some causal discovery algorithms, including DECI, initial hyperparameter settings can make or break the results.

Finding good values of such hyperparameters might be challenging, in particular when we don't have a relevant benchmark dataset. We will discuss this challenge in greater detail in the last section of this chapter.

Keeping this in mind, let's move forward and see how to implement DECI.

We will learn about more hyperparameters on the way.

DECI in code

Let's implement DECI to get a more concrete understanding of how it works. The code for this chapter can be found in the accompanying notebook (<https://bit.ly/causal-ntbk-14>).

Let's start with the imports.

First, we'll import the `dataclass` decorator and NumPy and NetworkX libraries:

```
from dataclasses import dataclass
import numpy as np
import networkx as nx
```

The `dataclass` decorator will help us make sure that the model configuration is immutable and we don't change it by mistake somewhere on the way. We'll use NumPy for general numerical purposes and NetworkX for graph visualizations.

Next, we'll import PyTorch, PyTorch Lightning, and two convenient tools – `DataLoader` and `TensorDict`:

```
import torch
import pytorch_lightning as pl

from torch.utils.data import DataLoader
from tensordict import TensorDict
```

PyTorch is a popular deep learning framework and PyTorch Lightning (<https://bit.ly/IntroToLightning>) is a wrapper around PyTorch that aims at simplifying PyTorch's workflow (somewhat similar to what Keras does for TensorFlow). We won't make too much use of Lightning's capabilities in this chapter, but we'll leverage its convenient `.seed_everything()` method in order to set the random seed over different libraries.

Next, we'll import a familiar set of objects from gCastle:

```
from castle.datasets import DAG, IIDSimulation
from castle.common import GraphDAG
from castle.metrics import MetricsDAG
```

We'll use them to generate the data, plot the results, and compute useful metrics.

Next, we'll import a number of modules and methods from Causica. Causica (<https://bit.ly/MicrosoftCausica>) is a Microsoft-managed open source library focused on causal machine learning. At the time of writing this chapter (Q2 2023), DECI is the only algorithm available in the library, yet the authors have informed us that other algorithms will be added with time.

First, we import the `distributions` module:

```
import causica.distributions as cd
```

The `distributions` module contains a broad range of objects related to model-specific probabilistic operations.

Next, we import the **Input Convex Graph Neural Network (ICGNN)** module, three objects related to the computations of the augmented Lagrangian loss, and a method that computes the *DAG-ness score*:

```
from causica.functional_relationships import ICGNN
from causica.training.auglag import AugLagLossCalculator, AugLagLR,
AugLagLRConfig
from causica.graph.dag_constraint import calculate_dagness
```

ICGNN (Park et al., 2022) is a graph neural network architecture that we'll use to recover the functional relationships between variables.

Finally, we import additional utilities:

- `VariableTypeEnum`, which stores variable type information (continuous, binary, or categorical)
- The `tensordict_shapes()` function, which allows us to easily get information about the shapes of variables stored within a `TensorDict` container

We have skipped these imports here to avoid excessive clutter (check the notebook for a full list of imports).

TensorDict

`TensorDict` is a PyTorch-specific dictionary-like class designed as a data-storing container. The class inherits properties from PyTorch tensors, such as indexing, shape operations, and casting to a device. `TensorDict` provides a useful abstraction that helps to achieve greater modularity.

We're done with the imports; let's prepare for experiments!

We'll start with a quick setup.

DECI training can be accelerated using GPU. The following line will check whether a GPU is available in your system and store the relevant information in the `device` variable:

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Let's set a random seed for reproducibility:

```
SEED = 11
pl.seed_everything(SEED)
```

We're ready to generate the data. We'll generate 5,000 observations from a simple scale-free graph:

```
# Generate a scale-free adjacency matrix
adj_matrix = DAG.scale_free(
    n_nodes=4,
    n_edges=6,
    seed=SEED
)

# Generate the simulation
dataset = IIDSimulation(
    W=adj_matrix,
    n=5000,
    method='nonlinear',
    sem_type='mim'
)
```

Here, we follow the same process that we used in the previous chapter:

- We generate a graph using gCastle's DAG object
- We generate the observations using the IIDSimulation object

We picked non-linear data generated using the multiple index model (Zheng et al., 2020). *Figure 14.1* presents our generated DAG.

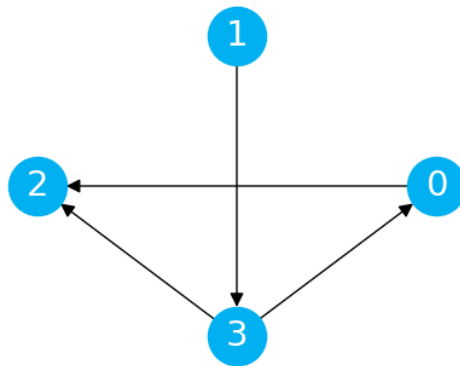


Figure 14.1 – Generated DAG

DECI configuration

As you have probably already noticed, DECI is a pretty complex model. In order to keep the code clean and reproducible, we'll define a set of neat configuration objects:

```
@dataclass(frozen=True)
class TrainingConfig:
    noise_dist=cd.ContinuousNoiseDist.SPLINE
    batch_size=512
    max_epoch=500
    gumbel_temp=0.25
    averaging_period=10
    prior_sparsity_lambda=5.0
    init_rho=1.0
    init_alpha=0.0

training_config = TrainingConfig()
auglag_config = AugLagLRConfig()
```

We use the `@dataclass` decorator with `frozen=True` in order to make sure that we don't alter the configuration object somewhere along the way mistakenly.

We instantiate model configuration (`TrainingConfig()`) and optimizer configuration (`AugLagLRConfig()`) and assign them to variables.

I set the batch size to 512 as I noticed that larger batches work better for small graphs with DECI.

Preparing the data

The dataset that we generated is stored as NumPy arrays. As DECI uses PyTorch, we need to cast it to `torch.tensor` objects.

We'll simultaneously store the tensors in a dictionary and then encapsulate them in a `TensorDict` object:

```
data_tensors = {}

for i in range(dataset.X.shape[1]):
    data_tensors[f'x{i}'] = torch.tensor(dataset.X[:,
        i].reshape(-1, 1))

dataset_train = TensorDict(data_tensors,
    torch.Size([dataset.X.shape[0]]))
```


Let's move the dataset to the device (the device should be cuda if a GPU accelerator had been detected on your system, otherwise cpu):

```
dataset_train = dataset_train.apply(lambda t:
    t.to(dtype=torch.float32, device=device))
```

Finally, let's create a PyTorch data loader, which will take care of batching, shuffling, and smooth data serving during training for us:

```
dataloader_train = DataLoader(
    dataset=dataset_train,
    collate_fn=lambda x: x,
    batch_size=training_config.batch_size,
    shuffle=True,
    drop_last=False,
)
```

DECI and expert knowledge

Thanks to its flexible architecture, DECI allows us to easily inject prior knowledge into the training process.

Let's pick one edge (let's say edge $(3, 0)$) in our graph and pass a strong belief about its existence to the model's prior.

Figure 14.2 presents a plot of the true adjacency matrix with edge $(3, 0)$ marked in red:

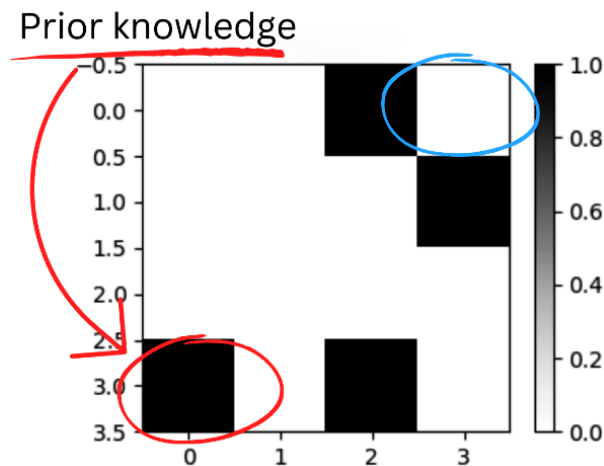


Figure 14.2 – The true adjacency matrix with prior knowledge edges marked

As you can see, there's also another spot marked in blue in *Figure 14.2*. This spot represents the same edge, but pointing in the opposite direction. As we are learning a directed graph (a DAG), the model should automatically understand that if the edge $(3, 0)$ exists, the edge $(0, 3)$ does not exist. The *DAG-ness penalty* in the cost function pushes the model toward this solution.

Although I haven't tested this systematically, it seems to me that explicitly passing the knowledge about the existence of edge $i \rightarrow j$ and the non-existence of edge $j \rightarrow i$ helps the model converge compared to the scenario where we only pass knowledge about the former.

In order to pass the knowledge to DECI, we need to build three matrices:

- An expert knowledge matrix
- A relevance mask that contains ones in the entries that we consider relevant and zeros everywhere else
- A confidence matrix that weights the relevant entries by our confidence, encoded as a float between 0 and 1

The relevance matrix is needed because we sometimes might want to pass our belief about the non-existence of an edge (encoded as 0 in the expert matrix) rather than its existence (encoded as 1 in the expert matrix).

The entries in the relevance matrix inform the model of which entries should be taken into account during the optimization.

Let's put all of this in the code.

First, we generate a zero matrix of the size of our adjacency matrix and assign 1 to the entry $(3, 0)$, where we believe an edge exists:

```
expert_matrix = torch.tensor(np.zeros(adj_matrix.shape))
expert_matrix[3, 0] = 1.
```

Next, in order to get the relevance matrix, we clone the expert matrix (we want the entry $(3, 0)$ to be taken into account by the model, so we can just reuse the work we just did) and set the entry $(0, 3)$ to 1:

```
relevance_mask = expert_matrix.clone()
relevance_mask[0, 3] = 1.
```

Now, the `expert_matrix` object contains 1 in position $(3, 0)$, while the `relevance_mask` object contains ones in positions $(3, 0)$ and $(0, 3)$.

Finally, we clone the relevance matrix to obtain the confidence matrix:

```
confidence_matrix = relevance_mask.clone()
```

We want to tell the model that we're 100% sure that the edge (3, 0) exists.

The confidence matrix takes values between 0 and 1, so the ones in the (3, 0) and (0, 3) entries essentially tell the model that we're completely sure that these entries are correct.

As DECI is a generative model, after the convergence, we can sample from the distribution over the adjacency matrix.

In order to effectively pass the knowledge to the model, we need to pass all three matrices to Causica's `ExpertGraphContainer` object:

```
expert_knowledge = cd.ExpertGraphContainer(
    dag=expert_matrix,
    mask=relevance_mask,
    confidence=confidence_matrix,
    scale=5.
)
```

The last parameter that we pass to the expert knowledge container – `scale` – determines the amount of contribution of the expert term to the loss.

The larger the value of `scale`, the heavier the expert graph will be weighted when computing the loss, making expert knowledge more important and harder to ignore for the model.

The main modules of DECI

DECI is largely a modular system, where particular components could be replaced with other compatible elements. This makes the model even more flexible.

Let's define DECI's main modules.

We'll start with the DAG prior:

```
prior = cd.GibbsDAGPrior(
    num_nodes=len(dataset_train.keys()),
    sparsity_lambda=training_config.prior_sparsity_lambda,
    expert_graph_container=expert_knowledge
)
```

Let's unpack it!

We use the `GibbsDAGPrior` class to define the prior. We pass three parameters to the class constructor: the number of nodes in the graph (which also represents the number of features in our dataset), the sparsity lambda value (this is the λ parameter, which weights the *sparsity score* that we discussed earlier in this chapter), and – last but not least – the expert knowledge object.

The Gibbs prior object will later be used in the training in order to compute the unnormalized log probability of the DAG that we'll use to compute the value of the loss function.

Next, let's build three components representing three elements of a **structural equation model (SEM)**:

- The adjacency matrix distribution module
- The functional module (which models the functional relationships between variables)
- The noise distribution module (which models the noise term distributions in the SEM)

We'll model the adjacency matrix using the `ENCOAdjacencyDistributionModule` object. ENCO (Lippe et al., 2022) is a causal discovery model that is able to work with observational, mixed, and interventional data. Contrary to many other algorithms, ENCO parametrizes the existence of an edge and its direction separately, and here we *borrow* this parametrization from the ENCO algorithm:

```
adjacency_dist = cd.ENCOAdjacencyDistributionModule(  
    num_nodes)
```

`ENCOAdjacencyDistributionModule`'s constructor takes only one argument – the number of nodes in a graph.

Next, we define the functional model. We'll use the ICGNN graph neural network (Park et al., 2022) for this purpose:

```
icgnn = ICGNN(  
    variables=tensordict_shapes(dataset_train),  
    embedding_size=8,  
    out_dim_g=8,  
    norm_layer=torch.nn.LayerNorm,  
    res_connection=True,  
)
```

We pass five parameters here:

- A dictionary of variable shapes (`variables`)
- The embedding size used to represent the variables internally (`embedding_size`)
- The size of the embeddings that represent the *parent* nodes while computing the representations of the *children* nodes internally (`out_dim_g`)
- An optional layer normalization object (`norm_layer`; <https://bit.ly/LayerNorm>)
- A Boolean variable telling the model whether we want residual connections to be used in the internal neural network or not (`res_connection`)

ICGNN will be responsible for parametrizing the functional relationships between the variables in our model.

Finally, let's define the noise module.

We'll start by creating a type dictionary. For each variable, we'll create a key-value pair with the variable name used as a key and its type as a value. We'll use Causica-specific type descriptions stored in the `VariableTypeEnum` object:

```
types_dict = {var_name: VariableTypeEnum.CONTINUOUS for
               var_name in dataset_train.keys() }
```

As all the variables in our dataset are continuous, we use the same type (`VariableTypeEnum.CONTINUOUS`) for all variables.

Finally, let's create a set of noise modules for each of the variables:

```
noise_submodules = cd.create_noise_modules(
    shapes=tensordict_shapes(dataset_train),
    types=types_dict,
    continuous_noise_dist=training_config.noise_dist
)
```

We pass the variable shapes and types to the constructor alongside the intended noise distribution.

The information about the noise distribution type is stored in our training configuration object. At the time of writing, DECI supports two noise distribution types: Gaussian and spline. The latter is generally more flexible and has been demonstrated to work better across a wide range of scenarios (Geffner et al., 2022, p. 7), and so we've picked it here as our default type.

Now, let's combine per-variable submodules into a joint noise module:

```
noise_module = cd.JointNoiseModule(noise_submodules)
```

We now have all three SEM modules (adjacency, functional, and noise) prepared. Let's pass them to a common SEM super-container and send the whole thing to a device:

```
sem_module = cd.SEMDistributionModule(
    adjacency_module=adjacency_dist,
    functional_relationships=icgcn,
    noise_module=noise_module)
sem_module.to(device)
```

The SEM module is now ready for training. The last missing part is the optimizer. DECI can use any PyTorch optimizer. Here, we'll use Adam.

First, we'll create a parameter list for all modules and then pass it to Adam's constructor:

```
modules = {
    'icgcn': sem_module.functional_relationships,
    'vardist': sem_module.adjacency_module,
    'noise_dist': sem_module.noise_module,
}
```

```
parameter_list = [
    {'params': module.parameters(), 'lr':
     auglag_config.lr_init_dict[name], 'name': name}
    for name, module in modules.items()
]
optimizer = torch.optim.Adam(params=parameter_list)
```

As we mentioned at the beginning of this section, DECI uses a constrained optimization scheme of augmented Lagrangian. Let's instantiate a learning rate scheduler and augmented Lagrangian loss calculator:

```
scheduler = AugLagLR(config=auglag_config)

auglag_loss = AugLagLossCalculator(
    init_alpha=training_config.init_alpha,
    init_rho=training_config.init_rho
)
```

We pass initial values of α and ρ to the `AugLagLossCalculator` constructor. They represent the coefficients α and ρ , which that we use to weight the *DAG-ness score* that we discussed earlier at the beginning of this section.

We're now ready to train the model!

Training DECI

In order to train DECI, we'll use a double `for` loop.

In the outer `for` loop, we'll iterate over the number of epochs, and in the inner `for` loop, we'll iterate over the batches within each epoch (as a result, we iterate over all batches within each epoch).

Before we start the `for` loops, let's store the total number of samples in our dataset in a `num_samples` variable. We'll use it later to compute our objective:

```
num_samples = len(dataset_train)

for epoch in range(training_config.max_epoch):
    for i, batch in enumerate(dataloader_train):
```

Within the loop, we'll start by zeroing the gradients, so that we can make sure that we compute fresh gradients for each batch:

```
optimizer.zero_grad()
```

Next, we'll sample from our SEM module and calculate the probability of the data in the batch given the current model:

```
sem_distribution = sem_module()
sem, *_ = sem_distribution.relaxed_sample(
    torch.Size([]),
    temperature=training_config.gumbel_temp
)
batch_log_prob = sem.log_prob(batch).mean()
```

Note that we use the `.relaxed_sample()` method. This method uses the Gumbel-Softmax trick, which approximates sampling from a discrete distribution (which is non-differentiable) with a continuous distribution (which is differentiable).

This is important because we cannot push the gradients through non-differentiable operations, which essentially makes training using gradient descent impossible.

Next, still within the batch loop, we compute the SEM distribution entropy (<https://bit.ly/EntropyDefinition>). We'll need this quantity to compute the overall value of the loss for the model:

```
sem_distribution_entropy =
    sem_distribution.entropy()
```

Next, we compute the log probability of the current graph, given our prior knowledge and the *sparsity score* we defined in the *DECI's internal building blocks* subsection earlier (the score is computed internally):

```
prior_term = prior.log_prob(sem.graph)
```

Next, we compute the objective and *DAG-ness score*, and pass everything to our augmented Lagrangian calculator:

```
# Compute the objective
objective = (-sem_distribution_entropy -
             prior_term) / num_samples - batch_log_prob

# Compute the DAG-ness term
constraint = calculate_dagness(sem.graph)

# Compute the Lagrangian loss
loss = auglag_loss(objective, constraint / num_samples)
```

We compute the gradients for the whole model and propagate them back:

```
loss.backward()
optimizer.step()
```

Finally, we update the augmented Lagrangian scheduler, which performs the augmented Lagrangian optimization procedure:

```

scheduler.step(
    optimizer=optimizer,
    loss=auglag_loss,
    loss_value=loss.item(),
    lagrangian_penalty=constraint.item(),
)

```

This concludes our training loop (in fact, in the notebook, we have a couple more lines that print out and plot the results, but we've skipped them here to avoid stacking too much code in the chapter).

Let's take a look at the results!

DECI's results

Figure 14.3 presents the recovered adjacency matrix (left) and the true matrix (right):

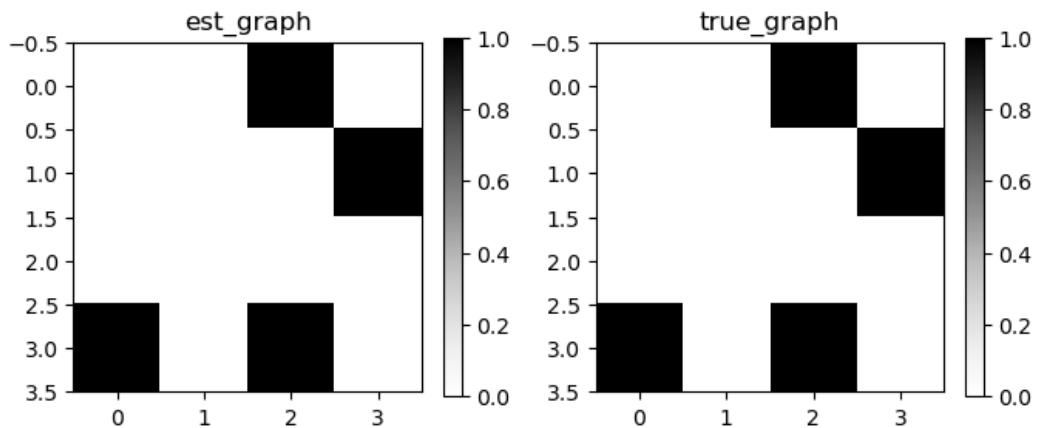


Figure 14.3 – The matrix recovered by DECI and the true matrix

As we can see, DECI did a very good job and recovered the matrix perfectly.

That said, we need to remember that we made the task easier for the model by providing some very strong priors.

I also found out that a number of hyperparameter settings were crucial for making the model results more or less stable for this and similar datasets. I used my knowledge and intuitions from previous experiments to choose the values.

First, I set the embedding sizes for ICGNN to 8. With larger, 32-dimensional embeddings, the model seemed unstable. Second, I set the batch size to 512. With a batch size of 128, the model had difficulties converging to a good solution.

The DECI architecture is powerful and flexible. This comes at the price of higher complexity. If you want to use DECI in practice, it might be a good idea to first work with the model on other datasets with a known structure that are similar to your problem in order to find good hyperparameter settings.

DECI also comes with a number of limitations – it is designed to work with non-linear data with additive noise, and when the data does not follow these assumptions, the model loses its theoretical guarantees (which are asymptotic).

Besides this, DECI requires standard assumptions of no hidden confounding, **minimality**, and general correct model specification.

For details on assumptions, check out Geffner et al. (2022), *Section 3.2*. For theoretical guarantees and their proofs, check out Geffner et al. (2022), *Theorem 1* and *Appendix A*.

Before we conclude this section, let's briefly discuss a capability of DECI that we haven't touched upon so far.

DECI is end-to-end

We used DECI as a causal discovery method, but in fact, it is an end-to-end causal framework, capable of not only causal discovery but also estimating the **average treatment effect (ATE)** and (to an extent) the **conditional average treatment effect (CATE)**.

In this section, we introduced DECI – a deep end-to-end causal inference framework. We discussed the basic theory behind the model and implemented it using Causica's modules and PyTorch. DECI is a flexible and powerful generative model that can perform causal discovery and inference in an end-to-end fashion.

Despite its many strengths, DECI – similar to the models that we discussed in the previous chapter – requires that no hidden confounding is present in the data.

Let's see what to do when the possibility of hidden confounding cannot be excluded.

Causal discovery under hidden confounding

Not all causal discovery methods are helpless in the face of hidden confounding.

In this section, we'll learn about the FCI algorithm, which can operate when some or all confounding variables are unobserved. We'll implement the FCI algorithm using the `causal-learn` package and – finally – discuss two more approaches that can be helpful when our dataset contains potential unobserved confounders.

The FCI algorithm

FCI (Sprites et al., 2000; Sprites et al., 2013) is a constraint-based algorithm. This means that the algorithm uses a set of conditional independence tests in order to decide which edges exist and what their orientations are. The FCI algorithm can be thought of as an extension of the PC algorithm that can work on an extended class of graphs, called **inducing path graphs**. The theory behind inducing path graphs is beyond the scope of our book. If you want to learn more, check out *Chapter 6* of Sprites et al. (2000).

I want more edge types, Mom!

FCI gives asymptotically correct results even under hidden confounding and selection bias.

This is great news!

However, in causality – similar to machine learning – there’s no free lunch, and there’s a price we need to pay for this otherwise-great feature. FCI (just like PC) might return a **Markov equivalence class (MEC)** rather than a fully oriented graph. In other words, some of the edges might not be oriented (even if they are correct). The algorithm also requires the **faithfulness assumption** to be met.

That said, FCI output can be more informative than the standard PC output. The reason for this is that FCI returns more edge types than just simple directed and undirected edges.

To be precise, there are four edge types in FCI (we’re following the notation scheme used in the `causal-learn` package):

- When $G_{ij} = -1$ and $G_{ji} = 1$, then i is a cause of j
- When $G_{ij} = 2$ and $G_{ji} = 1$, j is *not* an ancestor of i
- When $G_{ij} = 2$ and $G_{ji} = 2$, then no set d -separates i and j
- When $G_{ij} = 1$ and $G_{ji} = 1$, then there is a hidden common cause of i and j

If this sounds a bit overwhelming to you, you’re definitely not alone!

Fortunately, `causal-learn` also gives a simple visual representation of these types of edges that can be printed out for any found graph (we’ll see this in a while).

Implementing FCI

As of the time of writing, FCI is not available in `gCastle`. We will use the implementation from another library – `causal-learn`. This library is a Python translation and extension of the famous `TETRAD` Java library and is maintained by the `CLear` group from Carnegie Mellon University, which includes Peter Sprites and Clark Glymour – the creators of the original PC and FCI algorithms.

Traditionally, we start with the imports:

```
from causallearn.search.ConstraintBased.FCI import fci
from causallearn.utils.PCUtils.BackgroundKnowledge import
BackgroundKnowledge
from causallearn.graph.GraphNode import GraphNode
```

We import the `fci` function, which implements the FCI algorithm and the `BackgroundKnowledge` and `GraphNode` classes, which will allow us to inject prior knowledge into the algorithm.

Now, let's create a confounded dataset:

```
N = 1000

q = np.random.uniform(0, 2, N)
w = np.random.randn(N)
x = np.random.gumbel(0, 1, N) + w
y = 0.6 * q + 0.8 * w + np.random.uniform(0, 1, N)
z = 0.5 * x + np.random.randn(N)

data = np.stack([x, y, w, z, q]).T
confounded_data = np.stack([x, y, z, q]).T
```

We generate a five-dimensional dataset with 1,000 observations and different noise distributions (Gaussian, uniform, and Gumbel).

We create two data matrices: `data`, with all variables, and `confounded_data`, with the missing variable, `w` (which is a common cause of `x` and `y`).

Figure 14.4 presents the structure of our dataset. The red node, `W`, is an unobserved confounder.

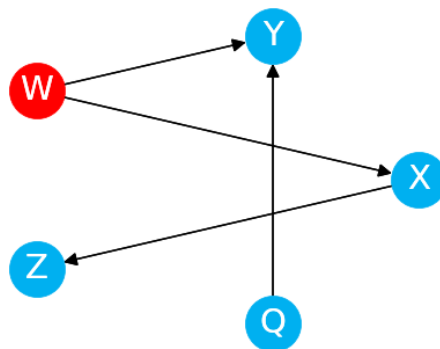


Figure 14.4 – A graph with an unobserved confounder

We're now ready to fit the model.

causal-learn's API is pretty different from the one we know from gCastle. First, the model is represented as a function rather than a class with a dedicated fitting method.

The model function (`fci()`) returns causal-learn's native graph object and a list of edge objects. Let's run the algorithm and store the outputs in variables:

```
g, edges = fci(
    dataset=confounded_data,
    independence_test_method='kci'
)
```

We passed the dataset to the function and, additionally, we specified the conditional independence test that we want to use. **Kernel-based conditional independence (KCI)** is a powerful and flexible test that can work with complex distributions. It scales well with the number of variables but can be very slow for large sample sizes.

Let's print the learned graph:

```
g.graph
```

This gives us the following:

```
array([[0, 2, 2, 0],
       [1, 0, 0, 1],
       [2, 0, 0, 0],
       [0, 2, 0, 0]])
```

For most people, this matrix is difficult to read (at least at first), and there's no easy way to meaningfully plot it using GraphDAG, which we've used so far.

Fortunately, we can iterate over the `edges` object to obtain a more human-readable representation of the results.

Let's create a mapping between the default variable names used internally by causal-learn and the variable names that we used in our dataset:

```
mapping = {
    'X1': 'X',
    'X2': 'Y',
    'X3': 'Z',
    'X4': 'Q'
}
```

Now, let's iterate over the edges returned by FCI and print them out:

```
for edge in edges:
    mapped = str(edge)\
        .replace(str(edge.node1),\
            mapping[str(edge.node1)])\
        .replace(str(edge.node2),\
            mapping[str(edge.node2)])
    print(mapped)
```

This gives us the following:

```
X o-> Y
X o-o Z
Q o-> Y
```

Let's decode it.

The meaning of the arrows in the printout is as follows:

- X is not an ancestor of Y – there might be an arrow from X to Y or they might be confounded, but there's no arrow from Y to X
- There's no set that d -separates X and Z – they might be confounded or have some direct causal relationship
- Y is not an ancestor of Q

Looking at the graph, all of this is true. At the same time, the output is not very informative as it leaves us with many unknowns.

FCI and expert knowledge

FCI allows us to easily inject expert knowledge into it. In causal-learn, we can add expert knowledge using the `BackgroundKnowledge` class. Note that we'll need to use the naming convention used by causal-learn internally (X_1 , X_2 , etc.) in order to specify expert knowledge (see the code snippet with the mapping dictionary in the preceding section for mapping between causal-learn's variable names and the variable names in our graph).

We instantiate the object and call the `.add_forbidden_by_node()` and `.add_required_by_node()` methods:

```
prior_knowledge = BackgroundKnowledge()
prior_knowledge.add_forbidden_by_node(GraphNode('X2'),
    GraphNode('X4'))
prior_knowledge.add_required_by_node(GraphNode('X1'),
    GraphNode('X3'))
```

In order to identify the nodes, we use `GraphNode` objects and pass node names that `causal-learn` uses internally (we could reverse our mapping here, but we'll skip it for simplicity).

Now, we're ready to pass our prior knowledge to the `fci()` function:

```
g, edges = fci(
    dataset=confounded_data,
    independence_test_method='fisherz',
    background_knowledge=prior_knowledge
)
```

Note that this time, we also used the Fisher's Z-test instead of KCI. Fisher-Z is a fast but slightly less flexible conditional independence test, compared to KCI. Fisher-Z is recommended for Gaussian linear data, but I've seen it work multiple times for much more complex distributions and relationships as well.

Let's see the results:

```
X o-> Y
X --> Z
Q --> Y
```

As we can see, disambiguating the relationship between Y (X_2) and Q (X_4) and directly enforcing the edge between X (X_1) and Z (X_3) worked well. The `-->` symbol reads *is a cause of*. The retrieved graph combined with expert knowledge is much more informative.

FCI might be a good choice if we're not sure about the existence of edges. The algorithm can give us a number of useful hints. The edges might next be passed in the form of expert knowledge to another algorithm for further disambiguation (e.g., LiNGAM or DECI if the respective assumptions are met). FCI performance can also be improved by using so-called tiers when passing background knowledge to the algorithm (**tiered background knowledge**). For details, check out Andrews et al. (2020).

Other approaches to confounded data

FCI is not the only method out there that can handle hidden confounding.

Another, more recent, solution is called the **confounding cascade nonlinear additive noise model (CCANM)**; Cai et al., 2021).

CCANM

CCANM is an **additive noise model (ANM-family)** model that leverages a **variational autoencoder (VAE)** in order to retrieve the causal direction between variables, including mediation and confounding scenarios. The authors have shown that the model outperforms models such as a basic ANM and LiNGAM on a number of benchmarks. CCANM tends to work better with larger sample size (5,000-6,000) observations.

CORTH

CORTH is an algorithm proposed by Ashkan Soleymani from MIT and colleagues (Soleymani et al., 2022). The algorithm uses the **double machine learning** framework to identify the direct causes of a variable of interest. It is framed by the authors as a *causal feature selection algorithm*. The algorithm allows for hidden confounding in the dataset, but *not* between the variable of interest and its direct causes.

CORTH requires that no node in the dataset is a descendant of the variable of interest, but it does not require faithfulness nor acyclicity between covariates and allows for non-linear interactions between them.

In this section, we introduced causal discovery methods that can work under hidden confounding. We discussed the FCI algorithm and implemented it using the causal-learn library. We also learned how to pass prior knowledge to the algorithm and introduced two other methods that can work under different scenarios involving hidden confounding: CCANM and CORTH.

In the next section, we'll discuss methods that can leverage information coming from interventions.

Extra – going beyond observations

In certain cases, we might be able to intervene on some or all variables in order to facilitate or improve the results of a causal discovery process.

In this short section, we'll introduce two methods that can help us make sure that we make good use of such interventions.

ENCO

Efficient Neural Causal Discovery (ENCO; Lippe et al., 2022) is a causal discovery method for observational and interventional data. It uses continuous optimization and – as we mentioned earlier in the section on DECI – parametrizes edge existence and its orientation separately. ENCO is guaranteed to converge to a correct DAG if interventions on all variables are available, but it also performs reasonably well on partial intervention sets. Moreover, the model works with discrete, continuous, and mixed variables and can be extended to work with hidden confounding. The model code is available on GitHub (<https://bit.ly/EncoGitHub>).

ABCI

Active Bayesian Casual Inference (ABCI; Toth et al., 2022) is a fully Bayesian framework for active causal discovery and reasoning. ABCI requires no hidden confounding or acyclicity and assumes a non-linear additive noise model with homoscedastic noise. A great advantage of ABCI is that it does not necessarily focus on estimating the entire causal graph, but rather on a causal query of interest, and then sequentially designs experiments that most reduce the uncertainty. This makes ABCI highly data-efficient. As a Bayesian method, ABCI makes it easy to encode expert knowledge in the form of a prior(s). Moreover, ABCI allows for different types of causal queries: causal discovery, partial causal discovery, SCM learning, and more.

The model code is available on GitHub (<https://bit.ly/ABCIGitHub>).

In this section, we introduced two methods: ENCO and ABCI. They allow us to combine observational and interventional data. ENCO provides excellent results when interventions for all variables are available (almost error-free on graphs of up to 1,000 nodes), while ABCI provides excellent data efficiency and can help focus efforts where it's most needed. Both frameworks significantly expand the realm of what's possible with causal discovery and can bring benefits even when only minimal interventions are possible.

Causal discovery – real-world applications, challenges, and open problems

Before we wrap up this chapter, let's take a broader perspective and discuss the applicability of causal discovery to real-world problems and challenges that may arise along the way.

In the previous chapter, we mentioned that Alexander Reisach and colleagues have demonstrated that the synthetic data used to evaluate causal discovery methods might contain unintended regularities that can be relatively easily exploited by these models (Reisach et al., 2021). The problem is that these regularities might not be present in real-world data.

Another challenge is that real-world data with a known causal structure is scarce. This makes synthetic datasets a natural benchmarking choice, yet this choice leaves us without a clear understanding of what to expect of causal structure learning algorithms when applied to real-world datasets.

The lack of reliable benchmarks is one of the main challenges in the field as of the time of writing this chapter.

That said, there exists research examining causal discovery methods' performance on real-world data.

We'll now briefly discuss it.

Tu et al. (2019) applied four traditional causal discovery algorithms (FCI, RFCI, PC, and GES) to real-world and (realistically) simulated data for neuropathic pain diagnosis. The dataset contains over 200 binary variables representing various injuries and symptoms.

The results for all causal discovery methods had limited quality in this setting. F1 scores varied roughly between 0.01 and 0.32, depending on the method and setting (top scores required a very large sample size, that is, >16,000 samples).

Huang et al. (2021) applied NO TEARS, DAG-GNN, and **Temporal Causal Discovery Framework (TCDF)** methods to climate data. The dataset consisted of a set of variables representing the interactions of Arctic Sea ice and the atmosphere. The authors concluded that they were able to achieve *reasonable results* (a normalized Hamming distance of around 0.3) compared to the expert graph with NO TEARS and DAG-GNN (but not TCDF).

The authors also report that both methods were sensitive to hyperparameter changes. This is challenging in a real-world setting, where the true graph is unknown and there's no good general benchmark, because we don't know which values of hyperparameters to choose.

Shen et al. (2020) applied FCI and FGES algorithms to Alzheimer's disease data. The FGES algorithm provided promising results with precision varying between 0.46 and 0.76 and recall varying between 0.6 and 0.74, depending on how much expert knowledge was injected into the algorithm.

The latter results show how valuable adding expert knowledge to the graph can be.

Causal discovery algorithms are also used in the industry. Unfortunately, most industrial use cases are protected by non-disclosure agreements and cannot be shared publicly.

I hope that with the rising adoption of causal methods in the industry, this will start gradually changing and we'll start seeing more companies sharing their experiences. This is of paramount importance as industrial use cases provide the research community with a vital injection of motivation. The research community gives back, providing the industry with better and more efficient ideas, which moves the industry forward.

We can observe the effects of such a virtuous circle in traditional machine learning today.

Benchmarking is not the only challenge in contemporary causal discovery.

Many real-world problems might be describable using mixed variable types (discrete and continuous). For most causal discovery methods, this is a major challenge.

ENCO supports mixed types natively, but only when we provide the algorithm with interventional data. The necessity to use interventional data might be a serious limitation in applying this algorithm in certain use cases.

Contemporary causal discovery can be a source of valuable insights and can definitely be helpful, yet it should be used with the awareness of its limitations.

We still haven't reached the stage of a *fully automated scientist*, but I believe that causal discovery research can move us a bit closer toward this milestone.

Understanding which methods can work under which circumstances is crucial and I hope that this book gave you a solid foundation to build such awareness.

Congratulations on reaching the end of *Chapter 14*! Let's summarize what we've learned.

Wrapping it up!

In this chapter, we introduced several methods and ideas that aim to overcome the limitations of traditional causal discovery frameworks. We discussed DECI, an advanced deep learning causal discovery framework, and demonstrated how it can be implemented using Causica, Microsoft's open source library, and PyTorch.

We explored the FCI algorithm, which can be used to handle data with hidden confounding, and introduced other algorithms that can be used in similar scenarios. These methods provide a strong foundation for tackling complex causal inference problems.

After that, we discussed two frameworks, ENCO and ABCI, that allow us to combine observational and interventional data. These frameworks extend our ability to perform causal discovery and provide valuable tools for data analysis.

Finally, we discussed a number of challenges that we face when applying causal discovery methods to real-world problems.

We are inexorably approaching the end of our journey.

In the next chapter, we'll summarize everything we've learned so far, discuss practical ideas of how to effectively apply some of the methods we've discussed, and see how causality is being successfully implemented across industries.

References

- Andrews, B., Sprites, P., & Cooper, G. F. (2020). *On the Completeness of Causal Discovery in the Presence of Latent Confounding with Tiered Background Knowledge*. International Conference on Artificial Intelligence and Statistics.
- Cai, R., Qiao, J., Zhang, K., Zhang, Z., & Hao, Z. (2021). *Causal discovery with cascade nonlinear additive noise models*. *ACM Trans. Intell. Syst. Technol.*, 6(12).
- Geffner, T., Antorán, J., Foster, A., Gong, W., Ma, C., Kıcıman, E., Sharma, A., Lamb, A., Kukla, M., Pawlowski, N., Allamanis, M., & Zhang, C. (2022). *Deep End-to-end Causal Inference*. arXiv.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2020). *Generative adversarial networks*. *Communications of the ACM*, 63(11), 139-144.
- Goudet, O., Kalainathan, D., Caillou, P., Guyon, I., Lopez-Paz, D., & Sebag, M. (2018). *Causal generative neural networks*. arXiv.
- Huang, Y., Kleindessner, M., Munishkin, A., Varshney, D., Guo, P., & Wang, J. (2021). *Benchmarking of Data-Driven Causality Discovery Approaches in the Interactions of Arctic Sea Ice and Atmosphere*. *Frontiers in Big Data*, 4.
- Kalainathan, D., Goudet, O., Guyon, I., Lopez-Paz, D., & Sebag, M. (2022). *Structural agnostic modeling: Adversarial learning of causal graphs*. arXiv.
- Kingma, D. P., Salimans, T., Jozefowicz, R., Chen, X., Sutskever, I., & Welling, M. (2016). *Improved Variational Inference with Inverse Autoregressive Flow*. *Advances in Neural Information Processing Systems*, 29.